

Corrected Document Output

Generated on: 2025-06-10 05:43:01

Data Clustering Methods 1 Introduction In daily life as well as in different fields of science, we encounter large, sometimes enormous volumes of information. A single glance is enough for humans to distinguish the shapes of objects of interest in a specific image. However, intelligent machines remain incapable of prompt and error-free object recognition due to the lack of universal algorithms applicable to every situation. The objective of data clustering is to partition a dataset into clusters of similar data. Objects in the dataset may include bank customers, figures in a photograph, or sick and healthy individuals. Humans can effectively group one-dimensional data, but three-dimensional data poses significant challenges. The problem intensifies with real-world datasets containing thousands to millions of samples. Thus, algorithms for automatic data clustering are essential. These algorithms produce a fixed structure of data partition, including cluster locations, shapes, and membership degrees of each sample to each cluster. Data clustering is complex due to arbitrary cluster shapes, sizes, and the typically unknown number of clusters. No existing algorithm universally handles all cluster shapes. When selecting a clustering algorithm, leverage domain knowledge.

- Homogeneity in clusters: Data within a cluster should be as similar as possible.
- Heterogeneity between clusters: Data across clusters should be as distinct as possible.

Similarity is often measured via distance metrics (e.g., Euclidean norm). Clusters are frequently represented by their central points. Different similarity measures yield varying cluster shapes. Since clustering lacks a "desired output signal," it is an unsupervised learning task. This chapter covers partitioning methods, clustering algorithms, and validity measures.

1.5.3.4.1.2.0.1, 3.3, 4.4, 5.0, 4, 6.5, 7.2, 8.4 (5.4) For example, in the event of sustained breaches the Commission should increase the percentage reduction every year. I think that this is a basis for strong operational cooperation. [1].

■ ■1■ ■2■ ■■■■■■ In medical diagnostics, rows represent symptoms and columns represent patients. Cluster centers are vectors $\mathbf{c}_i = [c_{i1}, \dots, c_{in}]$, $i = 1, \dots, K$. The number of partitions for n objects into K clusters is:

1.2.1: $(-1)^{n-K} \frac{n!}{K!} \sum_{i=1}^K \binom{n}{i} i^{K-1}$ Example 8.1 Partitioning 100 patients into 5 clusters yields 6.5×10^{17} possible partitions.

Hard Partition of the Cluster Each object belongs entirely to one cluster.

Conditions: $\sum_{i=1}^K u_{ij} = 1$, $u_{ij} \in \{0, 1\}$, $u_{ij} \cap u_{ik} = \emptyset$, $u_{ij} \cap u_{ik} \neq \emptyset$, $u_{ij} \in \{0, 1\}$. $\mathbf{U}$ is the partition matrix of the matrix. The partition matrix $\mathbf{U} \times \mathbf{X}$ encodes membership

degrees $\mu_{ij} \in \{0,1\}$: $\mu_{11} = 0.2, \mu_{12} = 0.1, \mu_{13} = 0.7$. $\sum_{j=1}^3 \mu_{1j} = 1$. Example: A hard partition matrix for 3 clusters: $\mu = \begin{bmatrix} 0.2 & 0.1 & 0.7 \\ 0.9 & 0.8 & 0.3 \\ 0.1 & 0.1 & 0.8 \end{bmatrix}$. Fuzzy Partition Conversely, objects belong to multiple clusters with degrees $\mu_{ij} \in [0,1]$. For example, in the refrigeration and air conditioning sectors there is a strong upward trend in the use of fluorinated gases due to the phase-out of ozone-depleting substances. In addition, some *Y. pestis* strains are capable of interfering with immune signaling (e. Fuzzy partition matrix for an outlier (x10): $\begin{bmatrix} 0.06 & 0.02 & 0.98 & 0.99 & 0.01 & 0.29 \\ 0.89 & 0.93 & 0.00 & 0.33 & 0.05 & 0.38 \end{bmatrix}$ Possibilistic Partition. No sum constraint on μ_{ij} , but $\exists \mu_{ij} > 0: 0 < \mu_{ij} \leq 1$. Example: Possibilistic partition for noise (x10):

$\begin{bmatrix} 0.01 & 0.02 & 0.52 & 0.39 & 0.87 & 0.03 \\ 0.01 & 0.02 & 0.52 & 0.39 & 0.87 & 0.03 \end{bmatrix}$ U= $\begin{bmatrix} 0.2 & 0.1 & 0.7 \\ 0.9 & 0.8 & 0.3 \\ 0.1 & 0.1 & 0.8 \end{bmatrix}$. This is why I believe this to be an excellent initiative that merits my support. $\mu_{ij} = \sqrt{\sum_{k=1}^n (\mu_{ik} - \mu_{jk})^2}$ Logic Minkowski, GPU $\mu_{ij} = \sqrt[p]{\sum_{k=1}^n (\mu_{ik} - \mu_{jk})^p}$ GPU1.1.3.1-1.2-2.3-3.4-4.5-2-4-3-1-8-2, <http://fetch.com>, <http://fetch.4.4>, <http://fetch.2>, and Hamming (binary variables). • Weighted Euclidean: $\mu_{ij} = \sqrt{\sum_{k=1}^n w_k (\mu_{ik} - \mu_{jk})^2}$

K-Means Clustering Algorithm: A Detailed Explanation of the K-MeANS is an unsupervised machine learning algorithm used for partitioning a dataset 1. It is one of the simplest and most widely used clustering techniques due to its efficiency and ease of implementation. **Rationale Behind K-Means Clustering** The goal of K-Means is to minimize the variance within each cluster while ensuring that each data point belongs to the most suitable cluster. The algorithm iteratively refines the cluster assignments based on the mean (centroid) of data points. **Key Concepts:** • **Centroid:** The center of a cluster, computed as the mean of all points assigned to that cluster, is the center of the subcluster.

• **Cluster Assignment:** Each data point is assigned to the nearest centroid. • • • • **Objective Function** K-Means minimizes the sum of squared distances between data points and their respective cluster centroids. The algorithm follows an iterative approach with the following steps: **Step 1: Initialize Cluster Centers** Choose K initial centroids, either randomly from the dataset or using a smarter initialization algorithm (e.g., K-Means++). **Step 2: Assign Data Points to the Nearest Centroid** Using the Euclidean Distance formula For each data point x_i , assign it to the nearest centroid using the Euclidean distance formula: $\sqrt{\sum_{k=1}^n (x_{ik} - \mu_{jk})^2} = \sqrt{\sum_{k=1}^n (x_{ik}^2 - 2x_{ik}\mu_{jk} + \mu_{jk}^2)}$ • x_{ik} represents the i -th feature of the index, and μ_{jk} represents the j -th feature of the k -th centroid. • Each data point is assigned to the cluster with the closest centroid. $\mu_{jk} = \argmin_k \sqrt{\sum_{i=1}^n (x_{ik} - \mu_{jk})^2}$ Github1.1.2.3. **Step 3: Compute New Centroids with the Mean of All Data Points in the Cluster:** Github2.2 After assigning all points, update each

cluster centroid with the mean of all data points in that cluster: $\mu_k = \frac{1}{n_k} \sum_{i \in C_k} x_i$ 1.3:
 Github1 C_k is the set of points assigned to cluster k . The number of points in cluster k is the number of nodes in cluster k . • K is the total number of clusters. Step 4:
 Repeat Until Convergence of Centroids is Reached Repeat steps 2 and 3 until centroids no longer change significantly or a pre-defined number of iterations is reached.

Mathematical Formulation of K-Means and its Objective Function (Objective Function) and its Function (Objective) K-means minimizes the within-cluster variance, also known as the sum of squared errors (SSE): $SSE = \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2$ $\mu_k = \frac{1}{n_k} \sum_{i \in C_k} x_i$ $x_i \in \mathbb{R}^d$ • x_i is a data point assigned to cluster k . • C_k is a cluster of cluster k . • C_k is the set of all points in cluster k . The algorithm iteratively refines cluster assignments to minimize SSE . The result is the following: Simple and easy to implement. Works well with compact, compact, spherical clusters. Sensitive to initial centroid selection (can lead to local minima). Assumes clusters are spherical and equally sized. Bisecting K-Means Clustering Algorithm Bisecting K-Means is a hierarchical variant of the standard K-Means algorithm that uses a divisive approach to clustering. Unlike traditional K-Means, which partition data into K clusters in one step, bisecting K-Means recursively split clusters into two (bi-sectioning) until the desired number of clusters is reached. This algorithm combines the advantages of both hierarchical clustering and K-Means: • Better clustering compared to standard clustering. • Better cluster quality compared to Standard K-Means. • Efficient computation compared to agglomerative hierarchical clustering. • Robustness in handling different cluster shapes and structures.

Rationale Behind Bisecting K-Means Comparison with Standard K-Means Algorithm Approach Strength Weakness Faster for small datasets. Can get stuck in local optima, sensitive to initialization. K-Means-Partitions the entire dataset into K clusters in one step. Hierarchical Clustering Builds a tree of clusters (bottom-up or top-down). Produces a hierarchical structure useful for visualization. Computationally expensive for large datasets. Bisecting K-Means Means K-MeSplits clusters iteratively into two sub-clusters. Better cluster quality than K-Means and more efficient than hierarchical clustering. Requires choosing the number of clusters manually. Algorithm Steps: The algorithm follows a top-down hierarchical approach: Figure 1.1. Start with all data in one cluster. Once 2.1.2. Repeat until K clusters are formed. O Choose the largest cluster to split. O Apply K-Means with $K=2$ to bisect the selected cluster. O Select the best split (based on the sum of squared errors, SSE). 3. Assign data points to final clusters. Mathematical Formulation of a Non-Objective Function (SSE - Sum of Squared Errors) At each step, we minimize the total within-cluster sum of squared errors (SSE). $SSE = \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2$ • K = number of

data points. • x_i = i -th data point. • μ_k = centroid of cluster k . • $d(x_i, \mu_k)^2$ = squared Euclidean distance between x_i and μ_k .

K-Means Bisection Step 1: To split a cluster, we apply K-Means with $K=2$. Initialize two centroids μ_1 and μ_2 randomly. 2. Assign points to the closest centroid. $\mu_k = \text{argmin}_{\mu_k} \sum_{x_i \in C_k} d(x_i, \mu_k)^2$. 3. Update centroids iteratively: $\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$ where $|C_k|$ is the number of points in cluster k . This is the ratio of 1:4. Repeat until convergence (centroids do not change significantly). Better Cluster Quality: More stable and less sensitive to initialization than standard K-Means.

Efficient for Large Datasets: Reduces the computational cost of hierarchical clustering. Flexible: Can control granularity of clusters easily. Like K-Means, the number of clusters must be predefined. Can Be Computationally Expensive? It still requires multiple K-Means runs. Object Sensitive to Cluster Selection, 1.0. Choosing the largest SSE cluster at each step may not always be optimal. K-Means++ Clustering Algorithm: Detailed explanation K-Means++ is an improved version of the K-Means clustering algorithm that provides a better initialization strategy for selecting the initial cluster centroids. The primary goal of K-Means++ is to reduce the chances of poor clustering due to bad centroid initialization in standard K-means.

Rationale Behind K-Means++ Problems with Standard K-Means Initialization In standard K-Means, centroids are initialized randomly, which can cause: • Poor Convergence: Randomly chosen centroids may lead to slow or failed convergence. • Suboptimal Clusters: Bad initial centroids can result in highly imbalanced clusters. The algorithm may get stuck in a poor solution due to bad initialization. How K-Means++ Improves Initialization K-Means++ selects the initial centroids in a more intelligent way by ensuring that: 1. The first centroid is chosen randomly from the dataset. 2.

Subsequent centroids are chosen with a probability proportional to their squared distance from the original centroid and/or their relative distance/distance from existing Centroids. 3. This ensures that centroids are well-separated, leading to faster convergence and better co-clustering quality. K-Means++ Algorithm Steps for

K-Means++ K-Means++ Algorithm Steps for

K-Means++ follows these steps: Step 1:

Choose First Centroid Randomly From the dataset • Pick the first centroid μ_1

randomly from the dataset. Step 2: Compute Distances for Next Centroid Selection •

For each remaining data point x_i , compute its squared distance from the closest

data point to the next pre-selected centroid: $1 = \min_{\mu_k \in C} d(x_i, \mu_k)^2$

• $d(x_i, \mu_k)$ is the Euclidean distance between the nearest already selected centroid

(μ_k) and the nearest previously selected centroids (μ_1, μ_2) in the data point. •

Step 3: Select Next Centroid Probabilistically •

Select the next centroid. μ_k randomly, but with a probability proportional to $d(x_i, \mu_k)^2$:

$$P(x) = \frac{1}{N} \sum_{i=1}^N \frac{1}{\sum_{j=1}^K d(x, c_j)^2}$$
 This means that points farther from existing centres have a higher chance of being selected.

Step 4.1.1 Repeat Until 1.2.0.0x2.1x2 Repeat steps 2.1 and 3 until 1.5.2x3.00.2 Repeat Steps have been chosen. Step 5: Run Standard K-Means • Use the selected centroids as the starting points for the standard K-Means algorithm.

Mathematical Formulation of Euclidean Distance to the nearest selected centroid
 Distance Calculation of the nearest chosen centroid
 For each data point x , compute the squared Euclidean distance to the next selected centroid:

$$d(x, c) = \min_{c \in C} \sum_{i=1}^n (x_i - c_i)^2$$
 where C is the set of currently chosen centroids.

Probability of Selecting Next Centroid with the Right Data Point
 Each data point x is selected as the next centroid with probability:

$$P(x) = \frac{d(x, c)^{-2}}{\sum_{i=1}^N d(x_i, c)^{-2}}$$
 This ensures diverse and well-separated centroid selection.

Better Initialization of Reduces the risk of poor cluster formation. Faster Convergence: Fewer iterations required compared to random initialization. **More Accurate Clusters:** Produces more natural and stable clusters.

Slightly More Expensive Initialization Than Initialization Does Not Guarantee Optimal Clustering: Still dependent on initial choice.

K-Medoids Clustering Algorithm (PAM: Partitioning All-Around Medoids) - A Detailed Explanation of K-Medoids (K-Means) and K-Mans (KMans)
 K-Medoid is a clustering algorithm closely related to K-Mean, but instead of using the mean data points (centroid) as the representative of a cluster, it uses medoids—actual data points that minimize the total dissimilarity between themselves and all other points in the cluster. The most common implementation of K-Medoids is the PAM (Partitioning Around Medoids) algorithm, developed by Kaufman and Rousseeu (1987).

Rationale Behind K-Medoids Clustering System (KMS)
 K-Means is highly sensitive to outliers since it uses the mean as the cluster center. K-Medoids, on the other hand, uses medoids, which are actual data points, making it more robust to outliers and more interpretable in real-world applications.

Key Concepts:

- **Medoid:** A representative point in a cluster that minimizes the sum of dissimilarities, such as the number of cells within the cluster.
- **Cluster Assignment:** Each data point is assigned to the medoid with the smallest probability of non-dissimilarity.
- • • • **Objective Function:** K-Medoids minimizes the sum of absolute differences (or any other unchosen distance metric) rather than squared differences like K-Means.

K-Medoids Algorithm Steps (PAM) [edit]
Step 1: Initialize Medoids in the dataset
 • Select K initial medoids randomly from the dataset.

Step 2: Assign Each Data Point to the Nearest Medoid Based on the Distance Metric
 Assign each data point to the closest medoid based on a chosen distance metric (e.g., Manhattan-Manhattan or Euclidean distance).

Step 3, Step 2, Swap and Optimize Medoids, Secure, Secure. For each non-medoid data point, compute the cost of swapping it with the medoid. • • • • If a swap reduces the total clustering cost, perform

the swap. • Repeat until no further swaps improve the clustering. Step 4: Repeat Until Convergence of Medoids • Stop when the medoids remain unchanged or a pre-defined number of iterations is needed to be re-reached. Mathematical Formulation of the New York-Distance Metric (New York Distance Metric) K-Medoids can use any distance metric, but a common choice is the Manhattan Distance: • $d_{\text{Manhattan}}(x, y) = \sum_{i=1}^n |x_i - y_i|$ • Alternatively, Euclidean Distance can be used: $d_{\text{Euclidean}}(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$ Objective Functionality of the GK-Medoids minimizes the sum of distances between each point and its assigned medoid: $J = \sum_{i=1}^n \sum_{j=1}^K d(x_i, m_j) \cdot \mu_{ij}$ • μ_{ij} is the number of clusters. • • • • S_j is the set of points assigned to cluster j .

• m_j is the medoid of cluster j . • $d(x_i, m_j)$ is the distance between point x_i and medoid m_j . 1.1.2.3.4.5. Cost of Swapping a Medoid to Replicate a Non-Medoid: For each non-medoid data point, compute the change in total distance if it replaces an existing medoid m_j . ($\Delta J = \sum_{i \in S_j} d(x_i, m_j) - \sum_{i \in S_j} d(x_i, x_k)$ • $\Delta J < 0$, and the swap is accepted. If $\Delta J < 1$, then swapping reduces the total clustering cost, and The swap is accept. Robust to outliers since medoids are actual data points. More interpretable since cluster centers are real data points. Computationally expensive compared to K-Means. Sensitive to the choice of K (requires domain knowledge or Elbow Method). C-Means Clustering Algorithm (Fuzzy C-Means - FCM) C-Means Clustering, commonly referred to as Fuzzy C-Means, is a soft clustering algorithm that allows data points to belong to multiple clusters with varying degrees of membership. Unlike K-Means, which forces each data point into precisely one cluster, FCM assigns each point a membership value between 0 and 1, representing its degree of belonging to different clusters. FCM is widely used in applications with fuzzy boundaries, such as image segmentation, medical diagnostics, and pattern recognition.

Rationale Behind FCM Limitations of K-Means • Hard Clustering: Each point belongs to exactly one cluster, which is unrealistic in many real-world scenarios. • Lack of Uncertainty Representation: No indication of how strongly a point belongs to a given cluster or to a particular assigned cluster. Advantages of FCM clustering Soft Clustering: Data points can belong to multiple clusters with different degrees of membership. Better Representation of Uncertainty: Fuzzy membership functions help capture ambiguity in data. Improved Cluster Overlap Handling: Useful in cases where clusters have fuzzy boundaries. The Fuzzy C-Means (FCM) Iterative Algorithm Conversely, using fuzzy membership values, FCM minimises an objective function that measures the weighted sum of squared distances between data points and cluster centers. Algorithm Steps 1.1. Initialize K cluster centroids randomly. 2. Assign fuzzy membership values to each data point. 3. To compute new centroids based on the fuzzy memberships. 4. Update membership values iteratively. 1.5. Repeat until

convergence (change in centroids is small). Mathematical Formulation of FCM:
 Fuzzy-Objective Function (Fuzzy Objective Function) FCM minimizes the following fuzzy objective function:

$$J_m(\mathbf{U}, \mathbf{V}) = \sum_{i=1}^n \sum_{k=1}^K u_{ik}^m \cdot \|\mathbf{x}_i - \mathbf{c}_k\|^2$$
 where $\mathbf{x}_i$ is the i -th data point, u_{ik} is the membership of $\mathbf{x}_i$ in cluster k (fuzzy membership), $\|\mathbf{x}_i - \mathbf{c}_k\|$ is the Euclidean distance between $\mathbf{x}_i$ and $\mathbf{c}_k$, and m is the fuzziness parameter ($m > 1$), which controls the degree of fuzziness. A larger $m \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ makes clusters more fuzzy, while $m \rightarrow 6 \rightarrow 1$ approaches hard clustering. Membership Update Rule Annotations: Membership values are updated using:

$$u_{ik} = \frac{1}{\sum_{j=1}^K \left(\frac{\|\mathbf{x}_i - \mathbf{c}_k\|}{\|\mathbf{x}_i - \mathbf{c}_j\|} \right)^{\frac{2}{m-1}}}$$
 The denominator ensures that the sum of all membership values for a data point, perceptually equals 1.

• Estimate number of clusters. • $\mathbf{x}_i$ = i -th data point. 1.2.0.0, 0.2, 0-0.3, 0.64, 0.32, 0.65, 0.67, 0.66, 0.39, 0.37, 0.49, 0.43, 0.42, 0.46, 0.34, 0.68, 0.38, 0.36, 0.41, 0.53, 0.47, 0.51, 0.69, 0.31, 0.35, 0.33, 0.52, 0.26, 0.44, 0.48, 0.19, 0.27, 0.61, 0.29, 0.23, 0.21, 0.378, 0.22, • u_{ik} = membership of $\mathbf{x}_i$ in cluster k (fuzzy membership). • $\|\mathbf{x}_i - \mathbf{c}_k\|$ = Euclidean distance between $\mathbf{x}_i$ and $\mathbf{c}_k$.

$$\|\mathbf{x}_i - \mathbf{c}_k\| = \sqrt{\sum_{j=1}^D (x_{ij} - c_{kj})^2}$$
 $m = 1, 2, 3, 4, 5, 6, 7, 8, 9, 9$ • m = fuzziness parameter ($m > 1$), which controls the degree of fuzziness. A larger $m \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$ makes clusters more fuzzy, while $m \rightarrow 6 \rightarrow 1$ approaches hard clustering. Membership Update Rule Annotations: Membership values are updated using:

$$u_{ik} = \frac{1}{\sum_{j=1}^K \left(\frac{\|\mathbf{x}_i - \mathbf{c}_k\|}{\|\mathbf{x}_i - \mathbf{c}_j\|} \right)^{\frac{2}{m-1}}}$$
 The denominator ensures that the sum of all membership values for a data point, perceptually equals 1.

Soft Clustering: Points can belong to multiple clusters. Handles Overlapping Clusters: Suitable for real-world datasets with uncertain boundaries. Better than K-Means in Some Cases: Works well when clusters are not clearly separated. Computationally Expensive: More iterations due to fuzzy membership updates. Sensitive to Fuzziness Parameter (m): A poor choice of m can lead to bad clustering results. Note Does Not Guarantee Optimal Clusters? You can still get stuck in local minima. Possibilistic C-Means (PCM) Clustering Algorithm Possibilistic C-Means (PCM) is a clustering algorithm designed to address the limitations of Fuzzy C-Means (FCM). Unlike FCM, which assigns each data point a membership value that sums to 1 across all clusters, PCM allows data points to belong to clusters independently, meaning a data point can have a high membership in multiple clusters or none at all. PCM is especially useful in situations where outliers exist, or when the data contains overlapping clusters, as it does not force every data point into one of the predefined clusters. Rationale Behind PCM's "Limitations of Fuzzy C-Means (FCM)":

- Constraint Issue: FCM enforces $\sum_{k=1}^K u_{ik} = 1$ for all data points, meaning every point must belong to some cluster.
- Outlier Sensitivity tends to assign small but nonzero membership values to outliers, making the model less robust.
- Clusters Can Merge: When clusters are highly overlapping, FCM tends to merge them rather than maintain distinct groups.

Advantages of PCM for Cluster-Independent Membership Values: Each point has a possibility degree for each cluster instead of a forced sum constraint.

Handles Outliers, PCM can assign very low membership values to outliers, making it more robust. Better Separation of Clusters: Works well in datasets with overlapping clusters. Mathematical Formulation of PCM is a great tool for understanding the mathematical properties of physical objects. Objective Function: Theoretically, the following is a good example of the mathematical formulation of a physical object. PCM minimizes the following possibilistic objective function:
$$J = \sum_{k=1}^K \sum_{i=1}^n (1 - u_{ik})^2 \|x_i - c_k\|^2 + \sum_{k=1}^K (1 - u_{kk})^2 \|c_k - c_k\|^2$$
 where K = number of clusters, i = i -th data point, c_k = centroid of cluster k , u_{ik} = possibilistic membership value in cluster k , $\|x_i - c_k\|$ = Euclidean distance between cluster k (c_k) and x_i ($\|x_i - c_k\|$) = geometric distance between x_i . For example, in the refrigeration and air conditioning sectors there is a strong upward trend in the use of fluorinated gases due to the phase-out of ozone-depleting substances. α_k = cluster-specific constant that controls membership strength. Membership and Centroid Update Rules for PCM (Updating Membership Values) The PCM membership function is different from FCM and is given by:

$$u_{ik} = \frac{1}{1 + \left(\frac{\|x_i - c_k\|}{\|x_i - c_{k'}\|} \right)^{\frac{1}{1 - \alpha_k}}} = 1 - \left(\frac{\|x_i - c_k\|}{\|x_i - c_{k'}\|} \right)^{\frac{1}{1 - \alpha_k}}$$
 α_k is independent for each cluster (does not sum to 1). α_k is computed as:
$$\alpha_k = \frac{\sum_{i=1}^n (1 - u_{ik})^2}{\sum_{i=1}^n (1 - u_{ik})^2 + \sum_{i=1}^n (1 - u_{ik'})^2}$$
 where α_k is a user-defined parameter (typically between 0.1 and 1).

Updating Cluster Centroids Using the Weighted Mean Centroids are updated using the weighted mean:
$$c_k = \frac{\sum_{i=1}^n u_{ik} x_i}{\sum_{i=1}^n u_{ik}}$$
 This ensures that centroids are influenced more by points with higher membership values. Handles Outliers. Well: Unlike FCM, PCM does not force every point into a cluster. Independent Membership: Each point has a possibility degree for each cluster rather than a forced sum constraint. Better Cluster Separation: Useful in applications like medical imaging, anomaly detection, and document clustering. Sensitive to Parameter α_k : Poor choices of α_k can lead to incorrect clustering. Computationally Expensive: More iterations are needed compared to FCM. Clusters Can Overlap: If α_k values are not well chosen, PCM may fail to clearly separate clusters.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) algorithm DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that identifies clusters based on the density of data points in a given space. Unlike K-Means and hierarchical clustering, DBSCAN does not require the number of clusters to be predefined and can discover clusters of arbitrary shape while distinguishing noise (outliers). It is particularly useful for: Detecting clusters of varying shapes and densities Detecting patterns of varying sizes and shapes Handling noise (outliers) effectively and efficiently Clustering large datasets efficiently Rationale Behind DBSCAN and DBSCAN Limitations Limitations of Other Clustering Methods: • K-Means: Assumes clusters are spherical and requires

the number of clusters K as input. • Hierarchical Clustering: computationally expensive and requires choosing a linkage method. • Fuzzy C-Means (FCM): Struggles with noise and overlapping clusters. Advantages of DBSCAN: Does not require specifying K (the number of clusters). Can detect clusters of arbitrary shape (e.g., elongated, curved). Handles noise/outliers naturally by labeling them as noise points. Works well on large datasets with spatial data. Mathematical Formulation of DBSCAN DBSCAN relies on two key parameters: • ϵ (epsilon): The radius within which points are considered neighbors. The minimum number of points required to form a dense region (a cluster).

Definitions Package Preferences 1.0.1. Core Point: A point p is a core point if at least $MinPts$ points (including p itself) are defined as: within its ϵ -neighborhood: $N_\epsilon(p) = \{q \in D \mid dist(p, q) \leq \epsilon\}$ where $dist(p, q)$ is the Euclidean distance. 2. Border Point: A point that is not a core point but lies within the neighborhood of a non-core point. 3. Noise Point (Outlier) is a point that is neither a core point nor a border point. DBSCAN Algorithm Steps 1.1. For each point p in the dataset: • If p is already classified, skip it. • Find all points within distance ϵ of p (neighbors). • If p is a core point (i.e., has at least $MinPts$ neighbors): • Create a new cluster and recursively add reachable points. • If p is a border point, assign it to the nearest cluster. • If p is a noise point, label it as an outlier. Identifies clusters of arbitrary shape. Automatically detects noise/outliers. Does not require specifying K like K-Means. Efficient for large spatial datasets. Bad choices can lead to poor clustering. Not suitable for high-dimensional data due to the curse of dimensionality. Very sensitive to density variation. If clusters have varying densities, the DBSCAN may fail.